

Python for AI engineers



Maya Malamud
Class October 2025
Lecture #2

Today's agenda

- ✓ Data types: booleans, tuples
- ✓ Use conditional logic to build decision-making code (if – else)
- ✓ Apply while & for loops and control flow
- ✓ Data types: set & dictionaries

Variables Types: Booleans

- **Python boolean** type is one of the built-in data types in Python, which represents one of the two values i.e. True or False.

Input: 1==1

Output: True

Input: 2<1

Output: False

<i>op</i>	<i>meaning</i>	True	False
<code>==</code>	<i>equal</i>	<code>2 == 2</code>	<code>2 == 3</code>
<code>!=</code>	<i>not equal</i>	<code>3 != 2</code>	<code>2 != 2</code>
<code><</code>	<i>less than</i>	<code>2 < 13</code>	<code>2 < 2</code>
<code><=</code>	<i>less than or equal</i>	<code>2 <= 2</code>	<code>3 <= 2</code>
<code>></code>	<i>greater than</i>	<code>13 > 2</code>	<code>2 > 13</code>
<code>>=</code>	<i>greater than or equal</i>	<code>3 >= 2</code>	<code>2 >= 3</code>

Comparisons with int operands and a bool result

Data types: booleans

type boolean

```
a = True  
type(a)
```

```
b = False  
type(b)
```



```
<class 'bool'>  
<class 'bool'>
```

Data types: booleans

Using function bool()

```
# Returns False as x is not equal to y
x = 5
y = 10
print(bool(x==y))

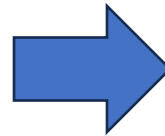
# Returns False as x is None
x = None
print(bool(x))

# Returns False as x is an empty sequence
x = ()
print(bool(x))

# Returns False as x is an empty mapping
x = {}
print(bool(x))

# Returns False as x is 0
x = 0.0
print(bool(x))

# Returns True as x is a non empty string
x = 'GeeksforGeeks'
print(bool(x))
```



Output

```
False
False
False
False
False
True
```

Data types: booleans

Comparing variables

```
# Declaring variables  
a = 10  
b = 20  
  
# Comparing variables  
print(a == b)
```



Output:

False

Data types: booleans

Integers and floats as booleans

Any integer, floating-point number, or complex number having zero as a value is considered as False, while if they are having value as any positive or negative number then it is considered as True.

```
var1 = 0
print(bool(var1))

var2 = 1
print(bool(var2))

var3 = -9.7
print(bool(var3))
```

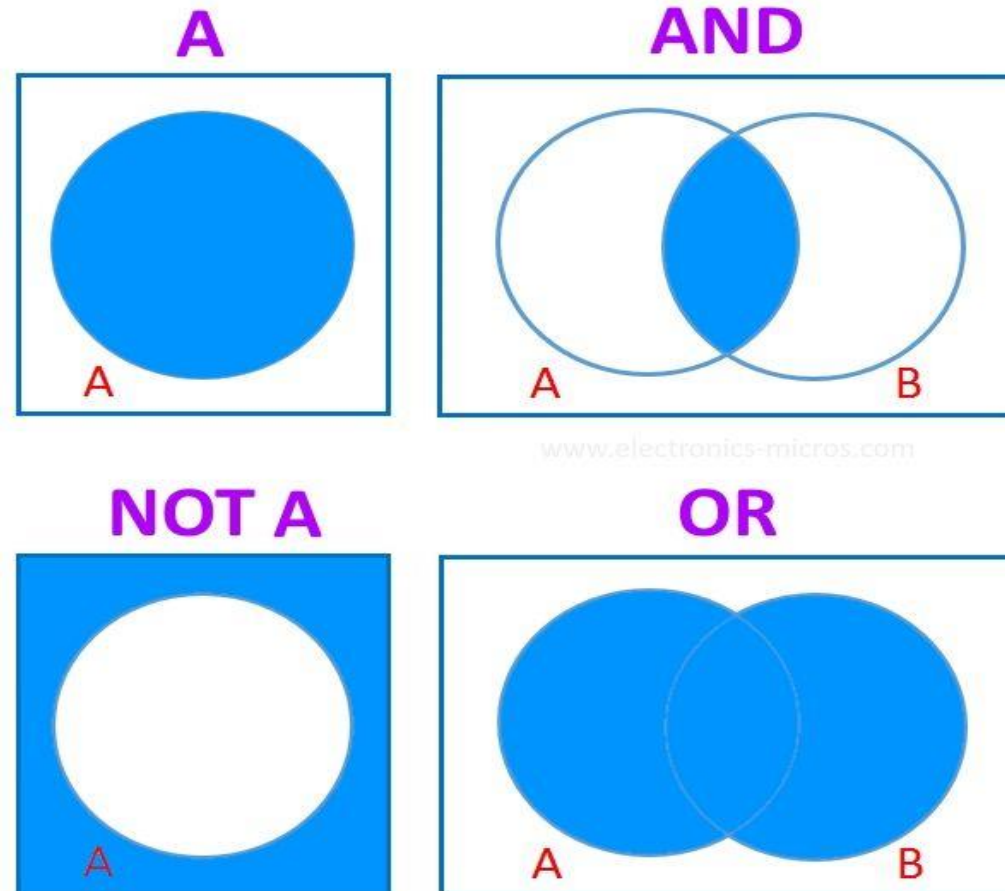


Output:

```
False
True
True
```

Boolean operators: OR, AND, NOT

Venn diagram



Data types: booleans

Boolean OR Operator

The Boolean *or* operator returns True if any one of the inputs is True, else returns False.

```
# Python program to demonstrate  
# or operator
```

```
a = 1  
b = 2  
c = 4
```

```
if a > b or b < c:  
    print(True)  
else:  
    print(False)
```

```
if a or b or c:  
    print("Atleast one number has boolean value as True")
```

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Output

```
True
```

```
Atleast one number has boolean value as True
```

Data types: booleans

Boolean AND Operator

The Boolean *and* operator returns False if any one of the inputs is False, else returns True.

```
# Python program to demonstrate
# and operator

a = 0
b = 2
c = 4

if a > b and b < c:
    print(True)
else:
    print(False)

if a and b and c:
    print("All the numbers has boolean value as True")
else:
    print("Atleast one number has boolean value as False")
```

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Output

```
False
Atleast one number has boolean value as False
```

Data types: booleans

Boolean NOT Operator

The Boolean *not* operator only requires one argument and returns the negation of the argument: returns True for False and False for True.

A	Not A
True	False
False	True

```
# Python program to demonstrate
# not operator

a = 0

if not a:
    print("Boolean value of a is False")
```

Output

```
Boolean value of a is False
```

Data types: booleans

Boolean == (equivalent) and != (not equivalent) operators

Both operators are used to compare two results:

== (equivalent) operator returns True if two results are equal

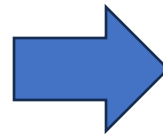
!= (not equivalent) operator returns True if the two results are not same.

```
a = 0
b = 1

if a == 0:
    print(True)

if a == b:
    print(True)

if a != b:
    print(True)
```



Output

```
True
True
```

Data types: booleans

is Operator

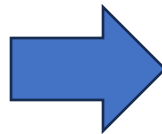
The test will return True if the two objects are the same, else it will return False even if the two objects are 100% equal.

```
x = 10
y = 10

if x is y:
    print(True)
else:
    print(False)

x = ["a", "b", "c", "d"]
y = ["a", "b", "c", "d"]

print(x is y)
```



Output

```
True
True
```

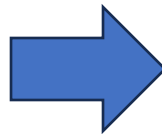
Data types: booleans in Operator

The in operator checks for the membership i.e. checks if the value is present in a list, tuple, range, string, etc.

```
# Python program to demonstrate
# in keyword

# Create a list
animals = ["dog", "lion", "cat"]

# Check if lion in list or not
if "lion" in animals:
    print(True)
```



?

Data types: lists

Example

List = [0, 1, 2, 3, 4, 5]

0	1	2	3	4	5
---	---	---	---	---	---

List[0] = 0

List[0:] = [0,1,2,3,4,5]

List[1] = 1

List[:] = [0,1,2,3,4,5]

List[2] = 2

List[2:4] = [2, 3]

List[3] = 3

List[1:3] = [1, 2]

List[4] = 4

List[:4] = [0, 1, 2, 3]

List[5] = 5

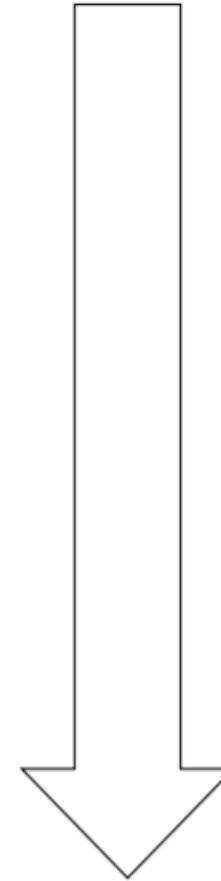
Data types: booleans

Example

What should this expression evaluate to?

```
3+4 == 14//2 and not 7 < 4.5
```





Data types: booleans

Example

What should this expression evaluate to?

`3+4 == 14//2 and not 7 < 4.5`

`7 == 14//2 and not 7 < 4.5`

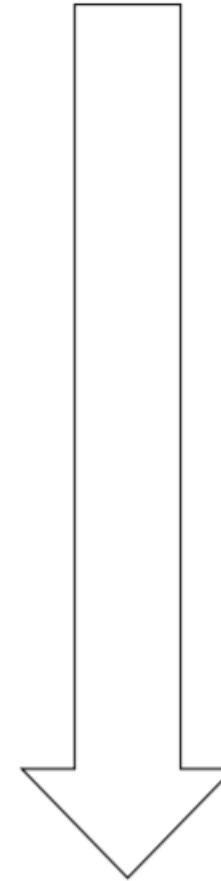
`7 == 7 and not 7 < 4.5`

`True and not 7 < 4.5`

`True and not False`

`True and True`

`True`



Data types: tuples

tup = (element1, element2,)

Tuples are similar to lists in many aspects, except:

- A tuple is a sequence of immutable (not-changeable) objects
- Tuples use parentheses, whereas lists use square brackets

Some Characteristics of Tuples:

- Tuples are immutable and ordered and allow duplicate values
- We can find items in a tuple since finding any item does not make changes in the tuple
- One cannot add items to a tuple once it is created
- Tuples cannot be appended or extended
- We cannot remove items from a tuple once it is created

Data types: tuples vs lists

Why choose a tuple over a list?

- Program execution is faster when manipulating a tuple than it is for the equivalent list
- If we don't want data to be modified

List	Tuple
1. List is mutable.	1. Tuple is immutable.
2. List iteration is slower and is time consuming.	2. Tuple iteration is faster.
3. List consumes more memory.	3. Tuples consumes less memory
4. List operations are more error prone.	4. Tuples operations are safe.
5. List provides many in-built methods.	5. Tuples have less in-built methods.
6. List is useful for insertion and deletion operations.	6. Tuple is useful for readonly operations like accessing elements.

Data types: tuples

Creating tuples using round brackets ()

```
var = ("Geeks", "for", "Geeks")  
print(var)
```



```
('Geeks', 'for', 'Geeks')
```

Data types: tuples

Creating tuples with one item

```
mytuple = ("Geeks",)
print(type(mytuple))

#NOT a tuple
mytuple = ("Geeks")
print(type(mytuple))
```



Output:

```
<class 'tuple'>
<class 'str'>
```

Data types: tuples

Tuples are immutable

```
list = [0, 1, 2, "a", "b"]  
mytuple = tuple(list)  
print(type(mytuple))
```

מדפיס טאפל

```
mytuple = (1, 2, 3, 4, 5)
```

```
# tuples are indexed  
print(mytuple[1])  
print(mytuple[4])
```

```
# tuples contain duplicate elements  
mytuple = (1, 2, 3, 4, 2, 3)  
print(mytuple)
```

```
# adding an element  
mytuple[1] = 100  
print(mytuple)
```

Output:

Python tuples are ordered and we can access their elements using their index values. They are also immutable, i.e., we cannot add, remove and change the elements once declared in the tuple, so when we tried to add an element at index 1, it generated the error.

```
2  
5  
(1, 2, 3, 4, 2, 3)  
Traceback (most recent call last):  
  File "e0eaddff843a8695575daec34506f126.py", line 11, in  
    tuple1[1] = 100  
TypeError: 'tuple' object does not support item assignment
```



Data types: tuples

Accessing tuples with positive indexing

```
var = ("Geeks", "for", "Geeks")  
  
print("Value in Var[0] = ", var[0])  
print("Value in Var[1] = ", var[1])  
print("Value in Var[2] = ", var[2])
```



Output:

```
Value in Var[0] =  Geeks  
Value in Var[1] =  for  
Value in Var[2] =  Geeks
```

Data types: tuples

Accessing tuples with negative indexing

```
var = (1, 2, 3)

print("Value in Var[-1] = ", var[-1])
print("Value in Var[-2] = ", var[-2])
print("Value in Var[-3] = ", var[-3])
```



Output:

```
Value in Var[-1] = 3
Value in Var[-2] = 2
Value in Var[-3] = 1
```


Data types: tuples

Concatenating tuples

```
# Code for concatenating 2 tuples
tuple1 = (0, 1, 2, 3)
tuple2 = ('python', 'geek')

# Concatenating above two
print(tuple1 + tuple2)
```



Output:

```
(0, 1, 2, 3, 'python', 'geek')
```

Data types: tuples

Creating nested tuples

```
# Code for creating nested tuples
tuple1 = (0, 1, 2, 3)
tuple2 = ('python', 'geek')

tuple3 = (tuple1, tuple2)
print(tuple3)
```



Output :

```
((0, 1, 2, 3), ('python', 'geek'))
```

Data types: tuples

Slicing tuples

```
# code to test slicing
tuple1 = (0 ,1, 2, 3)

print(tuple1[1:])
print(tuple1[::-1])
print(tuple1[2:4])
```



```
(1, 2, 3)
(3, 2, 1, 0)
(2, 3)
```

Data types: tuples

Deleting a tuple

Tuples are immutable; therefore, you cannot delete or reassign items from a tuple. But deleting a tuple entirely is possible by using the keyword *del*.

```
# Code for deleting a tuple  
tuple3 = ( 0, 1)
```

```
del tuple3  
print(tuple3)
```



Output:

```
Traceback (most recent call last):  
  File "d92694727db1dc9118a5250bf04dafbd.py", line 6, in <module>  
    print(tuple3)  
NameError: name 'tuple3' is not defined
```

Data types: tuples

Length of a tuple: len()

```
# Code for printing the length of a tuple  
tuple2 = ('python', 'geek')  
print(len(tuple2))
```



?

Data types: tuples

Converting into a tuple

```
# Code for converting a list and a string into a tuple
```

```
list1 = [0, 1, 2]
```

```
print(tuple(list1))
```

```
# string 'python'
```

```
print(tuple('python'))
```



```
(0, 1, 2)
```

```
('p', 'y', 't', 'h', 'o', 'n')
```

Data types: tuples

Counting elements in a tuple: count()

The count function can be used to count the number of occurrences of a specific element.

It is only used with lists and tuples.

```
a = [1,4,5,6,1]
```

```
b = (3,4)
```

```
2 1
```

```
print(a.count(1), b.count(3))
```

Data types: tuples

Examples



```
tup1 = ("We", "love", "learning", "Python")  
tup2 = (1, 2, 3)
```

```
print(tup1 + tup2)  
print(tup1, tup2)
```

```
('We', 'love', 'learning', 'Python', 1, 2, 3)  
('We', 'love', 'learning', 'Python') (1, 2, 3)
```

```
In [7]: tup=()
```

```
In [8]: type(tup)  
Out[8]: tuple
```

```
In [9]: tup = (2)
```

```
In [10]: type(tup)  
Out[10]: int
```

```
tup = (2,)
```

```
type(tup)  
tuple
```


Conditional Statements

- If-Else statements in Python are part of conditional statements, which decide the control of code.
- There are situations in real life when we need to explore a situation, and based on the findings, we decide what we should do next.
- Similar situations arise in programming: we need to explore a situation, and based on the findings, we will execute the next block of code, or not.

1.The if statement

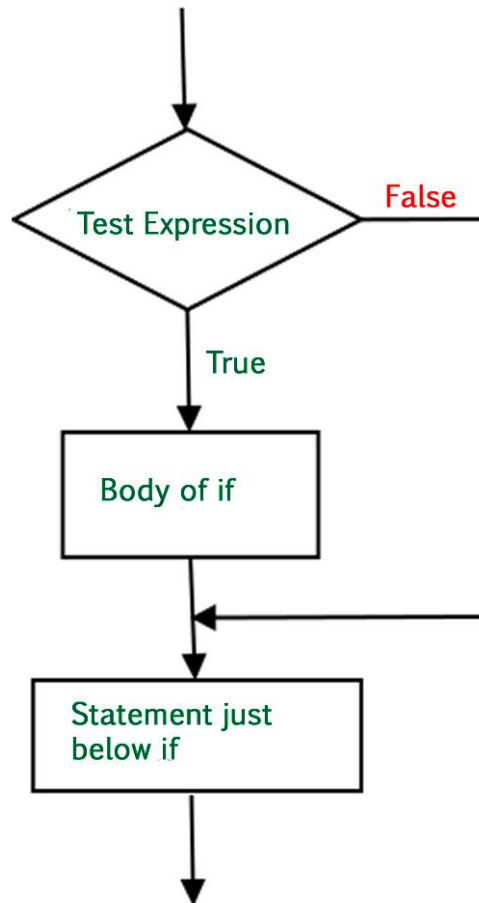
2.The if-else statement

3.The nested-if statement

4.The if-elif-else ladder

Conditional Statements

1. The if statement



```
x = 12
if x == 12:
    print("Yay!")

y = 4
if y == 4:
    print("Cool!")

z = True
if z == True:
    print("True")

if z:
    print("True")
```

Yay!
Cool!
True
True

Complex if statements



```
x = 12
if x == 12 and x > 0:
    print("Yay 1!")

if x >= 10 and x < 20:
    print("Yay 2!")

if 20 > x >= 10:
    print("Yay 3!")

if x == 25 or x > 10:
    print("Yay 4!")
```

Yay 1!
Yay 2!
Yay 3!
Yay 4!



```
x = 30

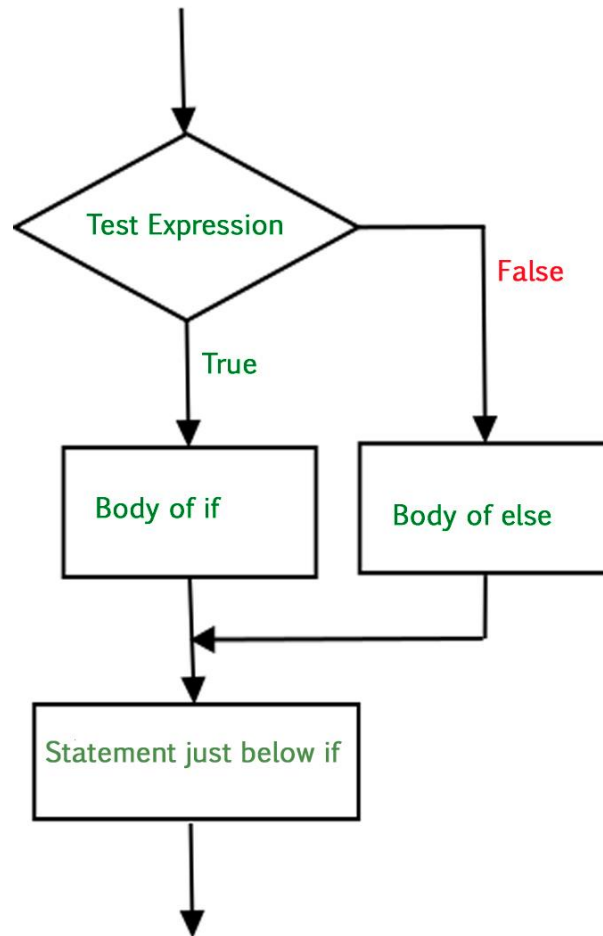
if x != 0:
    print("Nope 1!")

if x != None:
    print("Nope 2!")
```

Nope 1!
Nope 2!

Conditional Statements

2. The if-else statement



```
if case1():  
    do_this()  
else  
    do_that()
```

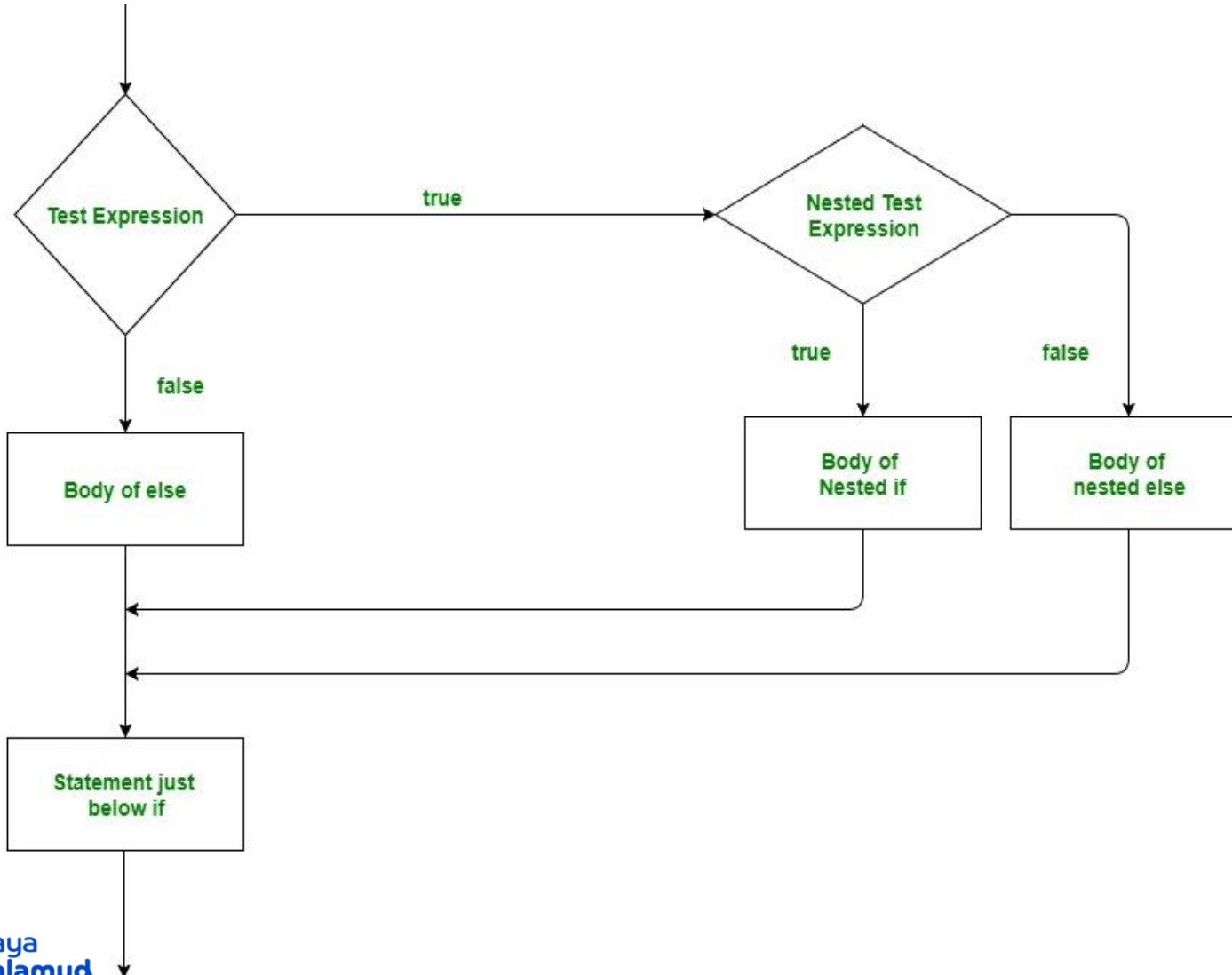


```
x = 10  
if x > 0:  
    print("Positive")  
elif x == 0:  
    print("Zero")  
else:  
    print("Negative")
```

Positive

Conditional Statements

3. The nested-if statement



Syntax:

```
if (condition1):  
    # Executes when condition1 is true  
    if (condition2):  
        # Executes when condition2 is true  
    # if Block is end here  
# if Block is end here
```

Conditional Statements

3. The nested-if statement: example

```
# python program to illustrate nested If statement
```

```
i = 10
```

```
if (i == 10):
```

```
    # First if statement
```

```
    if (i < 15):
```

```
        print("i is smaller than 15")
```

```
    # Nested - if statement
```

```
    # Will only be executed if statement above
```

```
    # it is true
```

```
    if (i < 12):
```

```
        print("i is smaller than 12 too")
```

```
    else:
```

```
        print("i is greater than 15")
```

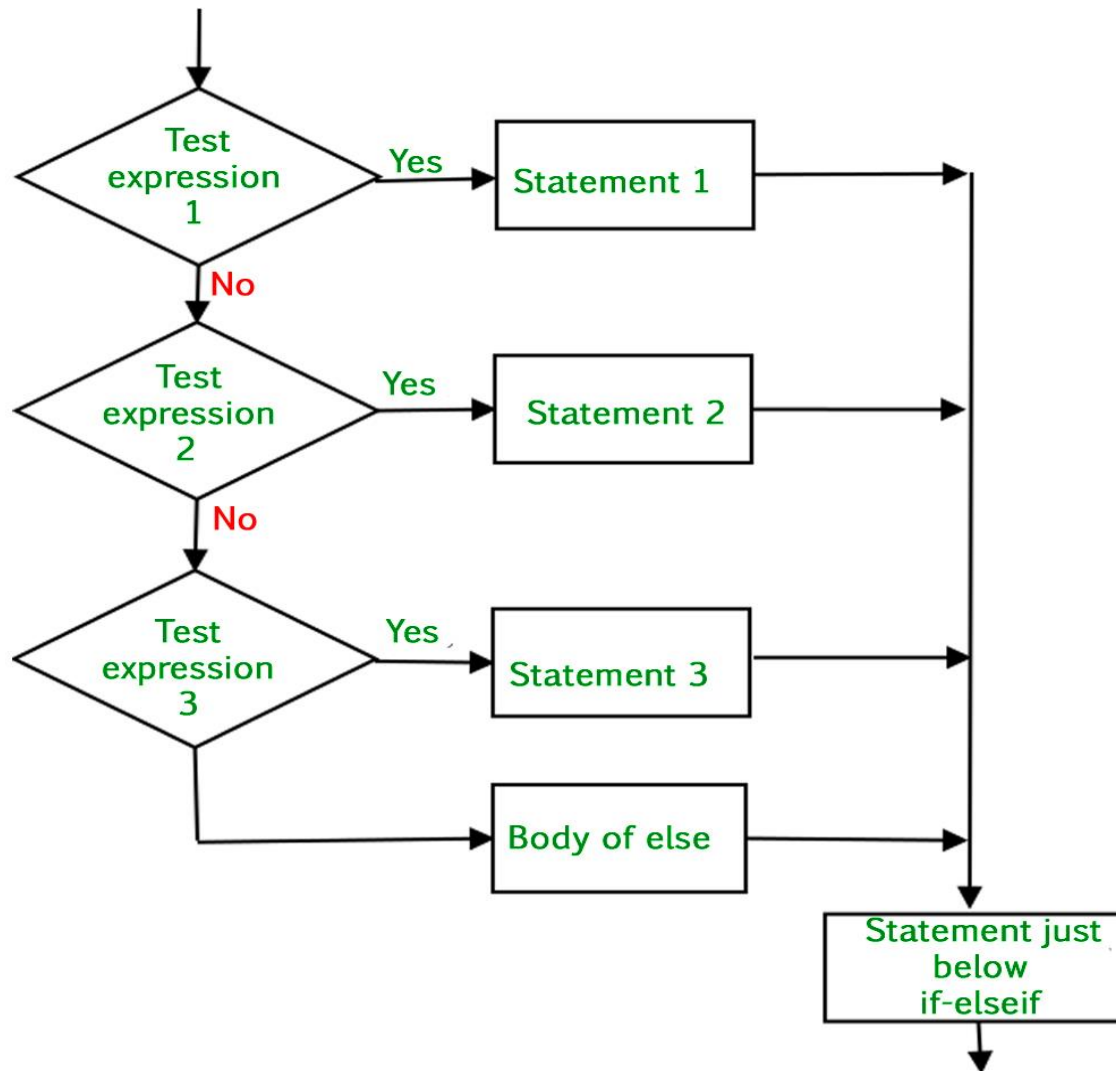
Output:

```
i is smaller than 15
```

```
i is smaller than 12 too
```

Conditional Statements

4. The if-elif-else ladder



Syntax:

```
if (condition):  
    statement  
elif (condition):  
    statement  
.  
.  
else:  
    statement
```

Conditional Statements

4. The if-elif-else ladder

```
i = 20
if (i == 10):
    print("i is 10")
elif (i == 15):
    print("i is 15")
elif (i == 20):
    print("i is 20")
else:
    print("i is not present")
```

Output:

```
i is 20
```


Conditional Statements

Shorthand if-else statement

```
i = 10
if i < 15: print("i is less than 15")
```

Output:

```
i is less than 15
```

```
# Python program to illustrate short hand if-else
i = 10
print(True) if i < 15 else print(False)
```

Output:

```
True
```

Conditional Statements

Indentation

```
x = 21
if x == 21:
    print("hooray")
    print ("cool")
    print("x is equal to 21")
```

```
x = 20
if x == 21:
    print("too bad")
    print("x is equal to 20")
    print("you got that")
```

```
x = 23
if x < 30 :
    print("Lower than 30")
    if x < 25:
        print ("Lower than 25")
        if x >20:
            print ("your getting closed")
            print("I")
            print("Love")
        print("python")
    print("you got that")
```

Function range()

The range() function return a sequence of numbers, starting from 0 by default, increment by 1 by default and stop before a given number

- `range(start, stop[, step])`
- The arguments to range() must be integers

```
In [6]: range(5)
```

```
Out[6]: range(0, 5)
```

```
In [7]: list(range(5))
```

```
Out[7]: [0, 1, 2, 3, 4]
```

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> list(range(0, 30, 5))
[0, 5, 10, 15, 20, 25]
>>> list(range(0, 10, 3))
[0, 3, 6, 9]
>>> list(range(0, -10, -1))
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
>>> list(range(0))
[]
>>> list(range(1, 0))
[]
```

Iterators - Loops

- Iteration: executing the same block of code over and over, potentially many times.
- Loop: A programming structure that implements iteration.
- An object is considered ***iterable*** if it is either a physically stored sequence (e.g. list, tuple, string, file), or an object that ***produces one result at a time*** in the context of an iteration tool like a for loop.
- Use case examples: reading files line by line, going over all rows in a table of data, optimizing a model by a list of parameters.

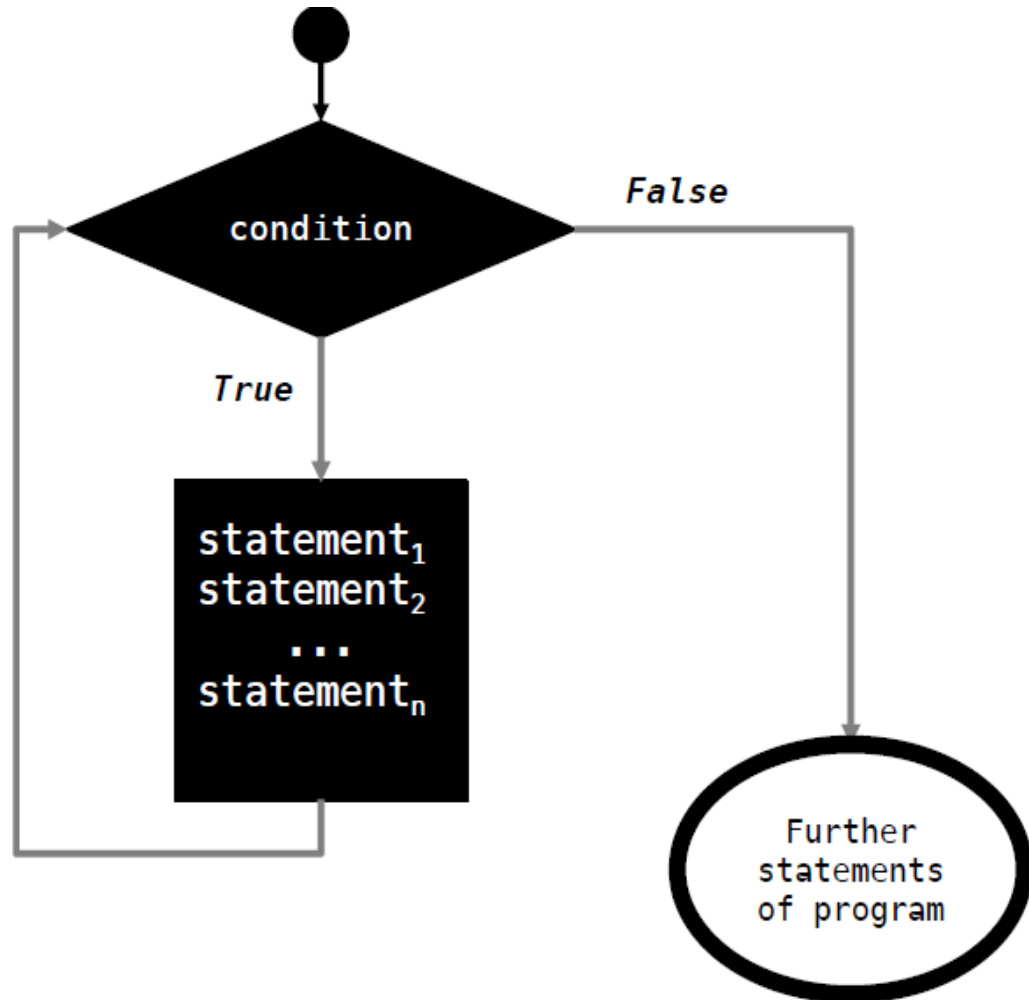
Iterators

While loop

Loops make it possible to carry out a sequence of statements repeatedly. The code within the loop, i.e. the code carried out repeatedly, is called the body of the loop

The while loop executes as long as the condition is true

```
While <condition>:  
    <statements>  
else: <statements>
```



Iterators

While loop: examples

```
n = 5
while n > 0:
    n -= 1
    print(n)
```

```
AGE = 16
while AGE < 17:
    AGE += 1 # equal to AGE = AGE + 1
    print("Not yet 17")
```

```
COUNT = 0
while COUNT < 5:
    print(COUNT)
    COUNT += 1 # equal to: COUNT = COUNT + 1
print("all done")
```

```
n = 0
while n > 0:
    n -= 1
    print(n)
```

Iterators

While loop: infinite loops

```
age = 28

# the test condition is always True
while age > 19:
    print('Infinite Loop')
```

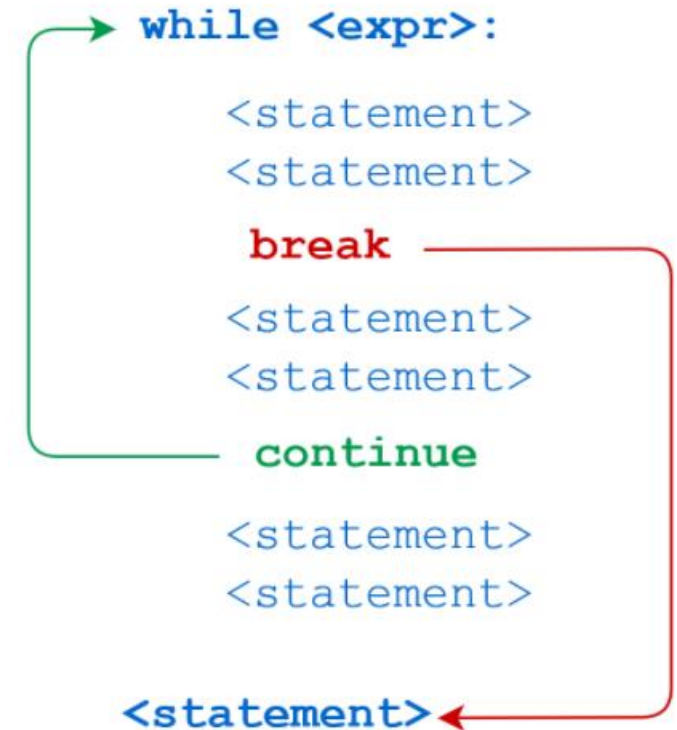
The value of the condition is always True.

Therefore, the body of the loop is run infinite times until the memory is full.

Iterators

Loop Control Statements

Control Statement	Description
break	Terminates the loop statement and transfers execution to the statement immediately following the loop
continue	Causes the loop to skip the remainder of its body and immediately retest its condition (or sequence) prior to reiterating
pass	<i>Null</i> operation; nothing happens when it executes. The pass is also useful in places where your code will eventually go, but has not been written yet



break and continue

Iterators

break

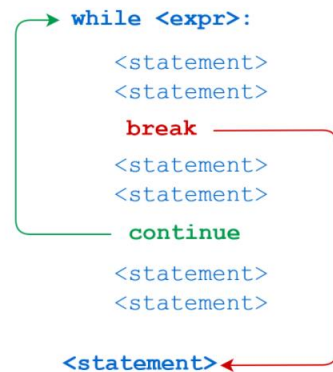
```
n = 5
while n > 0:
    n -= 1
    if n == 2:
        break
    print(n)
print('Loop ended.')
```

continue

```
n = 5
while n > 0:
    n -= 1
    if n == 2:
        continue
    print(n)
print('Loop ended.')
```

pass

```
n = 5
while n > 0:
    n -= 1
    if n == 2:
        pass
    print(n)
print('Loop ended.')
```



break and continue

Iterators

While-else loop

```
# Python program to demonstrate
# while-else loop

i = 0
while i < 4:
    i += 1
    print(i)
else: # Executed because no break in for
    print("No Break\n")
```

```
i = 0
while i < 4:
    i += 1
    print(i)
    break
else: # Not executed as there is a break
    print("No Break")
```

- The else clause is only executed when the while condition becomes false.
- If we break out of the loop, or if an exception is raised, it won't be executed.

Output

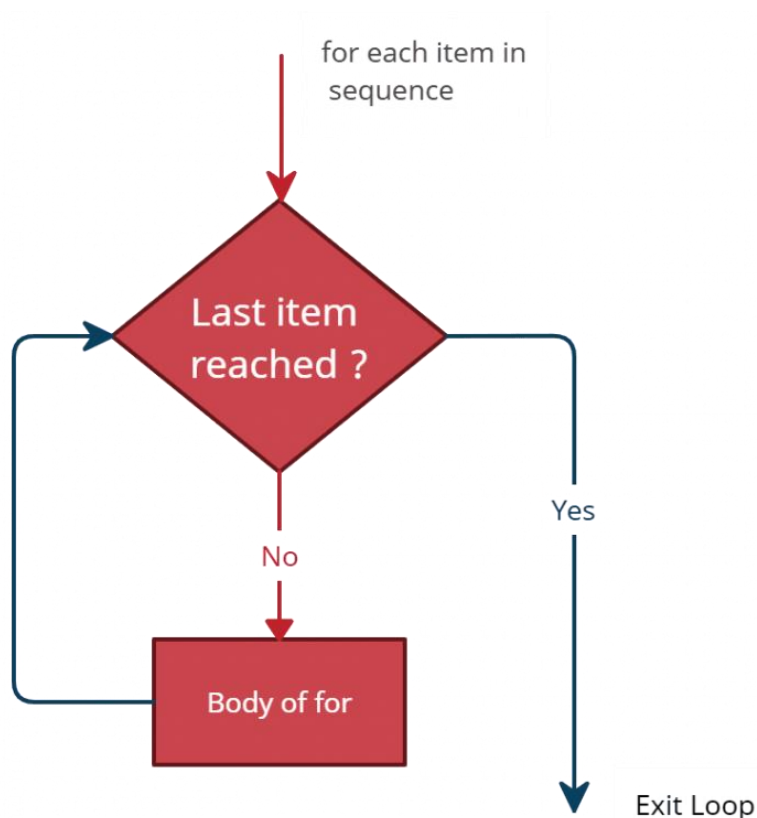
```
1
2
3
4
No Break

1
```

Iterators

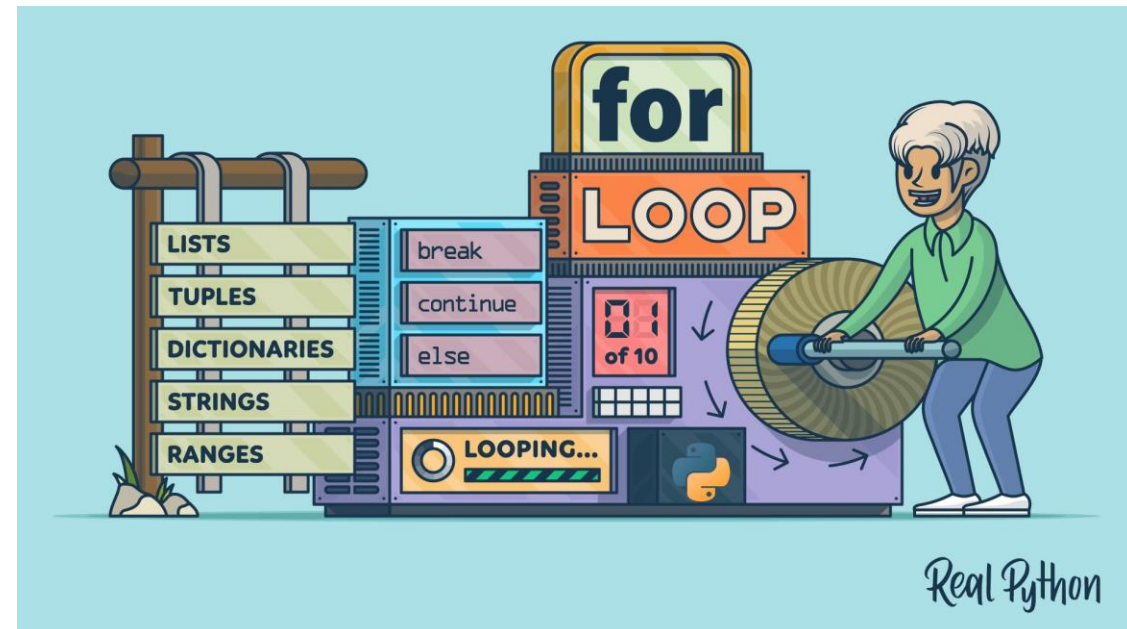
For Loop

For statement iterates over the items of any sequence (we have already met list, string, tuple, but this is also true for dict and iterators) in the order that they appear in the sequence.



```
In [20]: for number in range(5):  
...:     print(number)  
...:
```

0
1
2
3
4



Iterators

For loop with range()

Looping over a series of number: *range*(start, stop,[step])

```
In [4]: for i in range(5):  
....:     print(i)  
....:
```

0
1
2
3
4

```
In [5]: for i in range(3,15,3):  
....:     print(i)  
....:
```

3
6
9
12

Iterators

For loop: iterating over list and string

```
# Python program to illustrate  
# Iterating over a list  
l = ["geeks", "for", "geeks"]  
  
for i in l:  
    print(i)
```

Output :

```
geeks  
for  
geeks
```

```
# Iterating over a String  
print("String Iteration")  
  
s = "Geeks"  
for i in s:  
    print(i)
```

Output:

```
String Iteration  
G  
e  
e  
k  
s
```

Iterators

Nested For Loop

Nested loop is executed **for** each iteration of the outer **for**:

```
>>> combs = []
>>> for x in [1,2,3]:
...     for y in [3,1,4]:
...         if x != y:
...             combs.append((x, y))
...
>>> combs
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

This code will take 2 lists, and create a new list containing tuples, with all the combinations of the items from the first 2 lists, minus the ones where the items from the lists are identical.

Iterators

For Loop with Zip()

```
fruits = ["apple", "banana", "cherry"]  
colors = ["red", "yellow", "green"]  
for fruit, color in zip(fruits, colors):  
    print(fruit, "is", color)
```

Output :

```
apple is red  
banana is yellow  
cherry is green
```

- This code uses the zip() function to iterate over two lists (fruits and colors) in parallel.
- The for loop assigns the corresponding elements of both lists to the variables fruit and color in each iteration.
- Inside the loop, the print() function is used to display the message “is” between the fruit and color values.
- The output will display each fruit from the list of fruits along with its corresponding color from the colors list.

Data types: set

```
# Create an empty set:  
x = set()
```

- A set is similar to a list, but it cannot have multiple occurrences of the same element.
- A set contains an unordered collection of unique and immutable objects (after the set creation the objects become immutable).

```
a = {1,2,3,4}  
type(a)
```

set

{ } brackets

```
word = "apple"  
set(word)|
```

{'a', 'e', 'l', 'p'}

Calling set function
Keep unique values only

```
s = {1,2,3}  
s[0]
```

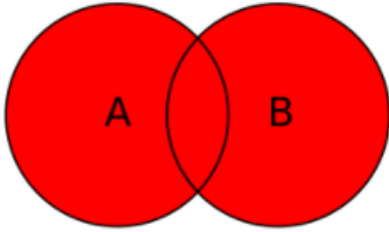
```
-----  
TypeError                                Traceback (most recent call last):  
<ipython-input-51-7bf80bf70831> in <module>  
      1 s = {1,2,3}  
>>> 2 s[0]  
TypeError: 'set' object is not subscriptable
```

Unordered
Slicing (by index) not possible

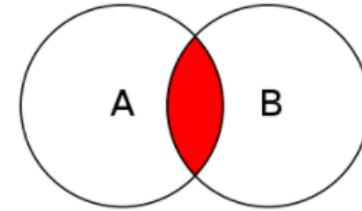
Venn diagram (set)

$A = \{1, 2, 3, 4, 5\}$

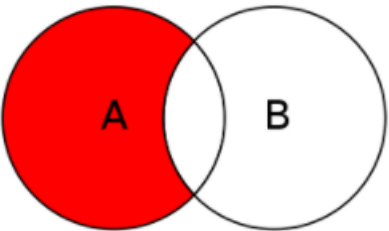
$B = \{6, 7, 5, 3, 8\}$



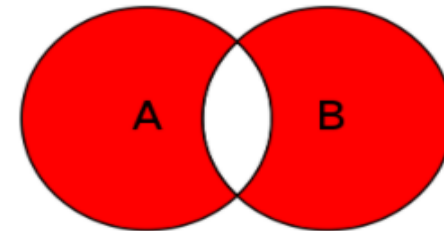
Union: $A \cup B = \{1, 2, 3, 4, 5, 6, 7, 8\}$



Intersection: $A \cap B = \{3, 5\}$



Difference: $A - B = \{1, 2, 4\}$



XOR: $A \oplus B = \{1, 2, 4, 6, 7, 8\}$

Operations on sets



```
basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
fruits = set(basket) # set() constructor
print(fruits) # unique items

print("apple" in fruits) # membership test
print("crabgrass" in fruits)
print("orange" not in fruits)
```

```
{'banana', 'apple', 'orange', 'pear'}
True
False
False
```



```
a = set('abracadabra')
b = set('alacazam')

print(a) # unique letters in a
print(b) # unique letters in b
print(a - b) # letters in a but not in b
print(a | b) # letters in a or b or both
print(a & b) # letters in both a and b
print(a ^ b) # letters in a or b but not both
```

```
{'c', 'r', 'd', 'b', 'a'}
{'c', 'm', 'l', 'z', 'a'}
{'d', 'b', 'r'}
{'c', 'm', 'l', 'r', 'd', 'z', 'b', 'a'}
{'c', 'a'}
{'d', 'b', 'm', 'r', 'l', 'z'}
```

Operations on sets

Operation	Equivalent	Result
<code>s.add(x)</code>		Adds an element <code>x</code> , which has to be immutable, to a set <code>s</code> if it is not already there.
<code>s.pop()</code>		removes and returns an arbitrary set element. The method raises a <code>KeyError</code> if the set is empty.
<code>s.discard(x)</code>		Removed <code>x</code> from set <code>s</code> . If <code>x</code> is not in <code>s</code> there is no change.
<code>s.remove(x)</code>		Removed <code>x</code> from set <code>s</code> . The method raises a <code>KeyError</code> if <code>x</code> is not in set <code>s</code> .
<code>s1.difference(s2)</code>	$s1 - s2$	Returns the difference of two or more sets as a new set
<code>s1.difference_update(s2)</code>	$s1 = s1 - s2$	Removes all elements of another set <code>s2</code> from set <code>s1</code> .
<code>s1.union(s2)</code>	$s1 \cup s2$	Returns the union of two sets as a new set, i.e. all elements that are in either set <code>s1</code> or <code>s2</code> .
<code>s1.intersection(s2)</code>	$s1 \cap s2$	Returns the intersection of set <code>s1</code> and set <code>s2</code> as a new set.
<code>s1.isdisjoint(s2)</code>		returns <code>True</code> if two sets have a null intersection
<code>s1.issubset(s2)</code>	$s1 \subseteq s2$; $s1 \subset s2$	Returns <code>True</code> if <code>s2</code> is a subset of <code>s1</code> . <code><</code> for proper subset.
<code>s1.superset(s2)</code>	$s1 \supseteq s2$; $s1 \supset s2$	Returns <code>True</code> if <code>s2</code> is a superset of <code>s1</code> . <code><</code> for proper superset.

Dictionary

```
# Create an empty dictionary:  
x = {}  
Or  
x = dict()
```

Create a dictionary explicitly with all its initial items

```
crime_and_punishment = {"Name" : "Crime and Punishment",  
                        "Author" : "Fyodor Dostoevsky",  
                        "Year": 1866,  
                        "Genre" : ["Philosophical novel", "Psychological fiction", "Crime  
fiction"],  
                        "My rating" : 4.5,  
                        "Audience rating" : 4.5}
```

- A dictionary stores key-value pairs of objects, separated by comma.
- A dictionary is defined using curly braces { }.
- The keys are immutable and appear at most once in the dictionary.
- The values can be any valid Python object.
- Keys could be a number, string, tuple etc.
- Values could be number, string, tuple, lists, dictionaries etc.



Dictionary examples (1)

```
a = dict(a=1,b=2,c=3,d=4)
a
{'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

Create a dictionary with function dict()

```
In [1]: # Creating a simple dictionary
dict1={101:'ram',
        102:'sham',
        103:'ravi'}

# printing the whole dictionary
print(dict1)

# accessing a value using its key
dict1[103]|

{101: 'ram', 102: 'sham', 103: 'ravi'}

Out[1]: 'ravi'
```

Create a dictionary explicitly with all its initial items

Dictionary examples (2)

```
students_grade = {}
```

Create a new empty dictionary

```
students_grade['00001234'] = 85
```

```
students_grade['00003579'] = 95
```

```
students_grade['00002468'] = 65
```

Set a grade for 3 students

```
students_grade = {'00001234':85, '00003579':95}
```

Recreate the same dictionary with only 2 of the students. The third student will no longer exist in it.

```
In [3]: students_grade["0002468"] = 77
```

Add the third student with a grade to the dictionary.

```
In [4]: students_grade
```

```
Out[4]: {'0001234': 85, '0003579': 95, '0002468': 77}
```

Operations on dictionary

Method	Description
<code>dict.get(key, default)</code>	For key <code>key</code> , returns value or default if key not in dictionary
<code>dict.setdefault(key, default)</code>	Similar to <code>get()</code> , but will set <code>dict[key] = default</code> if key is not already in dict
<code>dict.fromkeys(seq, value)</code>	Create a new dictionary with keys from <code>seq</code> and values set to <code>value</code> .
<code>dict.keys()</code>	Returns a view of dictionary <code>dict</code> 's keys
<code>dict.items()</code>	Returns a view of <code>dict</code> 's (key, value) tuple pairs
<code>dict.values()</code>	Returns a view of dictionary <code>dict</code> 's values
<code>dict.update(dict2)</code>	Adds dictionary <code>dict2</code> 's key-values pairs to <code>dict</code>
<code>dict.pop(key, [default])</code>	If <code>key</code> is in the dictionary, remove it and return its value, else return <i>default</i> . If <i>default</i> is not given and <code>key</code> is not in the dictionary, a <code>KeyError</code> is raised.

```
dict1={101:'ram',  
dict1.  
clear  
copy  
fromkeys  
get  
items  
keys  
pop  
popitem  
setdefault  
update
```

dict.setdefault()

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
  
x = car.setdefault("model", "Bronco")  
  
print(x)
```

Mustang

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
  
x = car.setdefault("color", "white")  
  
print(x)
```

white

```
[21] students = {'Alice': 21, 'Bob': 24, 'Charlie': 23}
```

```
age = students.setdefault('Alice', 22)  
print(age)
```

```
age = students.setdefault('David', 25)  
print(age)  
print(students)
```

21

25

{'Alice': 21, 'Bob': 24, 'Charlie': 23, 'David': 25}



dict.setdefault():

- If key exists, it will return the key's existing value.
- If the key doesn't exist, it will add the new key and its value to the dictionary.

Looping through dictionaries

By default, will print keys

```
for i in car:  
    print(i)
```

brand
model
year
color

Access values

```
for val in car.values():  
    print(val)
```

Ford
Mustang
1964
white

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
  
x = car.setdefault("color", "white")  
  
print(x)  
  
white
```

Get tuples of key, value

```
for item in car.items():  
    print(item)
```

('brand', 'Ford')
('model', 'Mustang')
('year', 1964)
('color', 'white')

Get key and value separately

```
for key, val in car.items():  
    print("the key is:", key + " " + "and the value is:", val)
```

the key is: brand and the value is: Ford
the key is: model and the value is: Mustang
the key is: year and the value is: 1964
the key is: color and the value is: white

Comma (,) will add a space and plus (+) won't

Group exercise – Lecture 2

Patient Records - You work in a small clinic that tracks basic test results for patients. You'll build and analyze simple records using Python.

1. Create Patient Records

Start with a list of **tuples**, each with a patient's name and age: `patients = [("Amit", 35), ("Dana", 17), ("Lior", 45), ("Sara", 22), ("Tom", 16)]`

2. Explore the Data

- Slice the list to keep only the **first 3 patients**.
- Create two separate lists: names and ages, using indexing or list comprehensions.
- Build a list `is_adult` with True/False values for whether each patient is **18 or older**.

3. Modify the Data

- **Remove** the second patient from the list using indexing.
- **Add** a new patient to the end.
- Use `bool()` to check if the list of names is non-empty.

4. Add Test Results (for the same patients)

- A list of `patient_ids`, like: `["P001", "P002", "P003", "P004", "P005"]`
- A list of `test_results`, where each item is a tuple like: `("Glucose", 88)`
- A dictionary that maps `patient_id` to the test result tuple.

5. Display Results

- Print each patient's ID, name, and test result using a **loop**.
- Only show results where the value is **above 90**.

6. Bonus (Optional)

- Create a **set** of all test names.
- Convert string dates (like `"2025-06-01"`) to real date objects using the code: `from datetime import datetime`
`date = datetime.strptime("2025-06-01", "%Y-%m-%d").date()`

Class exercise – suggested solution

```
# 1. Create Patient Records
patients = [("Amit", 35), ("Dana", 17), ("Lior", 45), ("Sara", 22), ("Tom", 16)]

# 2. Explore the Data
first_three = patients[:3]

# Create names and ages lists
names = [p[0] for p in first_three]
ages = [p[1] for p in first_three]

# Build list of booleans: is_adult
is_adult = [age >= 18 for age in ages]

# 3. Modify the Data, Remove second patient (Dana)
del patients[1]

# Add a new patient
patients.append(("Noa", 28))

# Check if names list is non-empty
print("Is the names list non-empty?", bool(names)) # Expected output: True
```

Is the names list non-empty? True

Test results above 90:

P002 - Lior: Glucose = 95

P004 - Sara: Glucose = 102

Unique test names: {'Glucose'}

Converted dates: [datetime.date(2023, 6, 1), datetime.date(2023, 6,

```
# 4. Add Test Results
patient_ids = ["P001", "P002", "P003", "P004", "P005"]
test_results = [("Glucose", 88), ("Glucose", 95), ("Glucose", 78), ("Glucose", 102), ("Glucose", 85)]

# Dictionary mapping patient ID to test result
id_to_result = dict(zip(patient_ids, test_results))

# 5. Display Results
print("\nTest results above 90:")
for pid, (name, _), result in zip(patient_ids, patients, test_results):
    test_name, value = result
    if value > 90:
        print(f"{pid} - {name}: {test_name} = {value}")

# 6. Bonus
# Create set of test names
test_names = {result[0] for result in test_results}
print("\nUnique test names:", test_names)

# Convert date strings to date objects
from datetime import datetime
date_strings = ["2025-06-01", "2025-06-02", "2025-06-03"]
date_objects = [datetime.strptime(d, "%Y-%m-%d").date() for d in date_strings]
print("Converted dates:", date_objects)
```